AWS Amplify

Explore how AWS Amplify enables developers to build and deploy full-stack web and mobile applications efficiently. Understand Amplify's components like authentication, data storage, and functions, and learn to use its CLI for backend provisioning. Discover seamless deployment with Amplify hosting and automated CI/CD pipelines, including support for feature branch deployments to streamline development and testing.

Amazon Amplify helps developers to quickly build, deploy, and host full-stack mobile and web applications on the AWS Cloud. It offers a comprehensive development framework that uses AWS services such as Cognito, AppSync, S3, and Lambda under the hood, enabling developers to focus on delivering features rather than managing infrastructure.

Imagine a developer tasked with delivering a full-featured MVP for a client in two weeks. We want rapid development but don't want to compromise on scalability, authentication, or data security. Traditionally, this would mean writing backend logic, configuring API endpoints, provisioning DynamoDB tables, setting IAM policies, managing Cognito user pools, and deploying everything manually.

Ask

Before getting into the details of application deployment with Amplify, let's first learn about its components.

Core components of AWS Amplify

AWS Amplify can be used to integrate AWS services into our application without worrying about managing these resources. It automatically provisions and connects cloud services like authentication, APIs, and storage, so we can build features without managing the underlying infrastructure. The following are some of the main components of AWS Amplify that use other AWS services to provide different capabilities to our application:

- **Amplify Auth:** AWS Amplify uses Amazon Cognito, which allows us to secure our application and allow users to sign into it easily. This allows us to easily authenticate and authorize our application's users.
- **Amplify Data:** AWS Amplify simplifies the process of storing, updating, and handling our web and mobile application data. It also provides us with DataStore, an on-device storage engine that can automatically synchronize data between our application and our database stored in the AWS cloud. It also provides easy-to-use APIs that can query, update, sort, and filter data in our applications. The APIs created by AWS Amplify are powered by AWS AppSync and connected to an Amazon DynamoDB database.
- **Amplify Storage:** AWS Amplify uses Amazon S3's capabilities to integrate file storage and management into our web and mobile applications seamlessly.
- **Amplify Functions:** Amplify Functions use AWS Lambda to perform various actions through self-contained functions. These functions can then be invoked whenever a certain event is generated in our application.

Ask

AWS Amplify AWS integrations

Now that we have learned about different components, let's learn how these can be used to deploy a complete full-stack application

Workflow of Amplify

Amplify's power lies in its CLI, which allows us to build a sophisticated backend directly from our project's terminal. Let's understand through the deployment of a simple application:

Ask

Step 1: Initialize the project

Start by installing the Amplify CLI and initializing it within the frontend application's directory.

main.sh Shell

```
npm install -g @aws-amplify/cli
amplify configure
amplify init
```

Command to initialize the project

The `init` command creates a local Amplify project that is linked to our AWS account and a corresponding backend environment in the cloud.

Step 2: Add features to the project

Instead of manually configuring AWS services, we add categories to our backend. Each category corresponds to a specific capability and uses a core AWS service under the hood. For example:

- **Authentication:** Running `amplify add auth` provisions an **Amazon Cognito** user pool, setting up sign-up, sign-in, and password recovery flows.
- **API:** `amplify add api` lets us create a GraphQL or REST API. GraphQL uses **AWS AppSync** and creates a **DynamoDB** database for us based on a simple schema file.

Ask

- **Storage:** `amplify add storage` sets up an **Amazon S3** bucket for user file uploads.
- **Functions:** `amplify add function` adds an **AWS Lambda** function for custom backend logic.

Step 3: Deploy the backend

Once we have defined our backend resources, we deploy them to the cloud with a single command:

main.sh Shell

```shell
amplify push
```

Command to deploy the backend

Amplify translates our local configuration into a CloudFormation template and provisions all the necessary resources in our AWS account. It also generates an `aws-exports.js` file, which our frontend application uses to connect to this new backend.

Ask

Amplify hosting and CI/CD

Amplify hosting is a fully managed CI/CD and hosting service specifically designed for modern web applications. It automates the entire process of deploying the frontend and connecting it to the Amplify-managed backend.

The CI/CD pipeline in Amplify hosting works as shown below:

Connect the Git repository

The process starts in the AWS Amplify console. First, we connect our project's source code from a Git provider like GitHub, Bitbucket or or GitLab. This connection enables Amplify to set up a continuous integration and continuous delivery (CI/CD) pipeline.

Configure build settings

Once the repository is connected, Amplify automatically detects the front-end framework used, such as React, Vue, or Next.js, and suggests build settings. These settings, which include commands for installing dependencies and building the app, are defined in an `amplify.yml` file. We can accept the defaults or customize this file to add tests or other stages to the build process.

Here is a sample `amplify.yml` file for a typical React application created with `create-react-app`.

index.yaml YAML

```yaml
version: 1
frontend:
  phases:
    # PRE-BUILD: Install dependencies
    preBuild:
      commands:
        - npm ci
    # BUILD: Run the build command
    build:
      commands:
        - npm run build
```

Ask

```
    # ARTIFACTS: Specify the output directory
```

Sample amplify.yml file

Let's understand the main sections of this file:

- `frontend`: It contains all the instructions for building the frontend application.
- `phases`: It defines the sequence of stages in the build process.
  - `preBuild`: It commands run before the main build, typically used for installing dependencies. `npm ci` is used for a clean, consistent installation.
  - `build`: The main command to compile the application. `npm run build` is the standard for `create-react-app`.
- `artifacts`: It tells Amplify where to find the built application files to be deployed.
  - `baseDirectory: build`: It specifies that the final output is in the `build` folder.
  - `files: ['**/*']`: It deploys all files within that `build` directory.
- `cache`: It specifies directories to cache between builds to improve performance. Caching `node_modules` avoids reinstalling all dependencies from scratch every time.

Deploy and host

After confirming the settings, Amplify kicks off the first deployment. This process has three stages:

- **Provision:** AWS sets up the necessary cloud infrastructure, including a globally available Content Delivery Network (CDN) to ensure the app l quickly for users anywhere in the world.

Ask

- **Build:** Amplify runs the build commands in a secure container, converting the source code into a deployable web application.
- **Deploy:** The built application is deployed to the hosting environment.

Once complete, the application is live on a secure `amplifyapp.com` domain.

Continuous deployment

From this point on, the deployment pipeline is fully automated. Every time we push a code change to the connected Git branch, Amplify will automatically trigger a new build and deploy the update. This seamless workflow allows developers to focus on developing features while Amplify handles the complexities of getting our app into the hands of our users.

Every subsequent `git push` to the connected branch automatically triggers a new build and deployment, creating a seamless CI/CD workflow.

Amplify branches

One of Amplify hosting's most powerful features is its support for feature branch deployments. When we connect a new branch from our Git repository, Amplify automatically creates a completely separate, independent deployment for it.

For example, consider that we have a `main` branch deployed to `https://main.ourapp.amplifyapp.com`. We want to live test a new feature in this application. To do so, we can follow the following simple steps:

1. We create a new feature branch in Git called `dev/new-login-flow`.
2. We push this branch to our repository.
3. In the Amplify Console, we connect this new branch.
4. Amplify automatically builds and deploys this branch to a unique URL `https://dev-new-login-flow.ourapp.amplifyapp.com`.

Ask

This allows our team to test and review new features in a live, production-like environment without impacting the main application. Amplify can also be configured to generate deployment previews for every pull request, further streamlining the review process.

Ask